IEEE INFOCOM 2026 - IEEE Conference on Computer Communications | 979-8-3315-4961-9/26/$31.00 ©2026 IEEE | DOI: 10.1109/INFOCOM59046.2026.11571199

QuIKS: Near-Zero Latency Key Supply with Adaptive Buffering for Resource-Efficient Quantum Key Distribution Networks

Yuxin Chen*, Zite Xia†, Jian Li†‖, Kaiping Xue†‡‖, Zhonghui Li§, Lutong Chen†, Ruidong Li¶
*Department of EEIS, University of Science and Technology of China, Hefei, Anhui 230027, China
†School of Cyber Science and Technology, University of Science and Technology of China, Hefei, Anhui 230027, China
‡Hefei National Laboratory, Hefei, Anhui 230088, China
§School of Electronic and Information Engineering, Anhui University, Hefei, Anhui 230601, China
¶National Institute of Information and Communications Technology, Kanazawa University, Tokyo 184-0015, Japan
‖Corresponding author: J. Li, K. Xue {lijian9, kpxue}@ustc.edu.cn

***Abstract*—Quantum key distribution (QKD) networks provide information-theoretically secure keys for distant parties, emerging as a vital alternative to classical cryptography infrastructures threatened by quantum computing. In QKD networks, the immediacy of key supply service is crucial to the security and performance of applications, as their data must be encrypted before transmission. While key buffering can enable instant key supply services, existing schemes rely on heuristic solutions that incur prohibitive key resource consumption, thus significantly hindering practical deployment. To address this issue, we propose QuIKS, an instant key supply scheme based on adaptive buffering, offering the dominant advantage of near-zero key supply latency while consuming ultra-low key resources (i.e., ultra-low buffer size). Specifically, it is built upon a novel analytical model that determines the minimum buffer size required to guarantee near-zero-latency key supply performance. Guided by this model, QuIKS introduces a lightweight two-phase control algorithm that dynamically determines key relaying requests and adjusts the buffer size by probing real-time application patterns and network conditions. Experiments on a real QKD network testbed demonstrate that QuIKS achieves near-zero key supply latency while providing a more than 10-fold reduction in key buffer size compared to state-of-the-art schemes.**

***Index Terms*—Quantum key distribution, quantum networks, end-to-end key supply, key buffering**

I. Introduction

Quantum key distribution (QKD) [1] is a cutting-edge technology that provides information-theoretically secure keys for two remote parties based on the principle of quantum mechanics [2], [3], making it a promising solution to defend against the threat posed by quantum computing to classical cryptography [4], [5]. To extend these security guarantees from point-to-point links to a large scale, QKD networks have been rapidly developed in recent years [6]–[9]. Among various architectural approaches, the trusted-relay technology emerges as the most mature and practical one for building large-scale QKD networks [10]. In this paradigm, a network of relay nodes is interconnected via individual links, in which each pair of adjacent nodes implements a QKD protocol to generate quantum keys. After that, the end-to-end key can be relayed along a multi-hop path through sequential encryption and decryption between adjacent nodes, consuming shared quantum keys.

The ultimate vision of QKD networks is to serve an on-demand key supply service, seamlessly integrating with applications that currently rely on classical cryptography [11]–[13]. However, the QKD network fundamentally struggles to provide this consistent service. On the one hand, quantum key generation, which determines the key resources between adjacent nodes, is inherently probabilistic and susceptible to environmental disturbances [14]. On the other hand, the underlying classical network, essential for key relaying, introduces its dynamics like traffic congestion and network updates [15], [16]. The instability leads to highly volatile network conditions for key relaying, which can cause significant service delays or even failures for secure applications, thereby constituting a major barrier to the widespread adoption of QKD networks.

To address this problem, various efforts have been made to enhance the performance of QKD networks, including sophisticated key management [17], [18], dynamic routing [19], [20], and resource allocation [21], [22]. Although these approaches mitigate key generation fluctuations, they still exhibit volatile performance due to heterogeneous paths or strategy switching, and thus, they hardly address the inherent performance fluctuations rooted in the underlying classical networks [23]. Given the storable nature of keys, a promising approach is to create end-to-end key buffers that can smooth out supply variations from any source. Existing buffering schemes fall into two categories: those that rely on static pre-allocation mechanisms [24], [25] and those that employ heuristic-based control [26]. While these schemes achieve desirable key supply latency, they also consume excessive link-level quantum keys from QKD networks to build up a large buffer size at end nodes. This results in significant key resource wastage, which cannot be ignored and severely hinders their practical application.

The inability of existing buffer-based schemes to strike an optimal trade-off between key supply performance and resource consumption stems from three fundamental challenges.

(1) The difficulty in modeling the impact of stochastic application requests and QKD networks. The key buffer state is governed by two unpredictable processes: the random arrival of application requests and the fluctuating key relaying process in QKD networks. This dual source of randomness makes it exceedingly difficult to accurately model and predict buffer state evolution. ***(2) The lack of quantitative evaluation for the performance-cost trade-off.*** Without a rigorous analytical model that connects key supply performance to buffer size, designing a buffer control mechanism becomes a matter of guesswork. This forces the existing schemes to adopt heuristic strategies with oversized key buffers as a safeguard against exhaustion, which inevitably leads to key resource waste and poor trade-offs. ***(3) The low-complexity requirement for the buffer control algorithm.*** In practice, the key buffer requires a computationally lightweight control algorithm to support real-time management of numerous concurrent key requests. This requirement creates a dilemma: heuristic strategies maintain computational efficiency but lack accuracy, while sophisticated models achieve high performance but are complex.

To tackle these fundamental challenges, we propose QuIKS, an **i**nstant **k**ey **s**upply scheme based on adaptive buffering with minimal **qu**antum key resource consumption. The core of QuIKS is an analytical model for the buffer-enabled end-to-end key supply process, aiming to characterize the impact of application requests and QKD networks by unifying both key generation and classical network fluctuations. Based on this model, we quantify the fundamental trade-off between buffer size and key supply performance using the central limit theorem under the wide-sense stationary assumption, providing theoretical guidance to escape the guesswork of heuristic-based strategies. Guided by the derived conclusions, we design a lightweight buffer control algorithm that dynamically adapts to application patterns and network conditions in a real-time manner, relaxing the stationary assumption. We then implement and evaluate QuIKS on a real-world QKD network testbed. Experimental results demonstrate that QuIKS achieves a near-zero key supply latency, while providing a more than 10-fold reduction in key buffer size compared to state-of-the-art schemes. By these efforts, QuIKS paves the way for the widespread adoption of QKD networks.

The contributions of this work are summarized as follows:

- For the first time, we propose a novel theoretical model for key supply service, which can quantify the mathematical relationship between buffer size and key supply performance, providing theoretical guidance for optimal buffer control design.
- We design a lightweight, theory-guided adaptive buffer control algorithm, named QuIKS, that translates our analytical insights into a practical mechanism for providing instant key supply service with ultra-low key resource consumption.
- We conduct extensive experiments on a real QKD network testbed, demonstrating that QuIKS reduces key buffer size by over 90% compared to state-of-the-art schemes while achieving near-zero key supply latency.

The rest of this paper is organized as follows. First, we present the background and our motivation in Sec. II. In Sec. III, we detail the system model and the analytic result. After that, we describe the QuIKS design in Sec. IV. Then, in Sec. V, we introduce the implementation and conduct experiments on a real testbed. Finally, we briefly review related work in Sec. VI and conclude our work in Sec. VII.

II. Background and Motivation

In this section, we first introduce the necessary preliminaries and then articulate our motivation with an experiment. After that, we present our design goals.

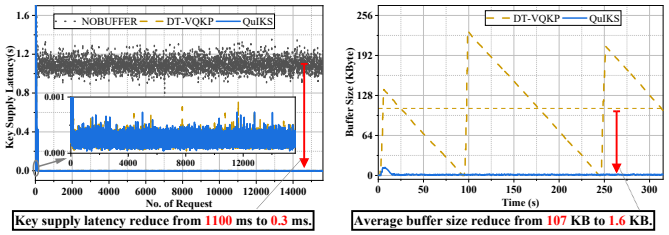
A. Key Supply Services in QKD Networks

QKD networks [6]–[9] are built to extend the range and scalability of various QKD protocols [27], [28], which are inherently limited by quantum signal attenuation in the physical channels. Functionally, the architecture of QKD networks is standardized into a three-layer framework [29]. The bottom layer comprises multiple independent point-to-point QKD protocols responsible for generating quantum keys between respective node pairs. The middle layer handles end-to-end key supply services for secure applications (e.g., IPSec [30] or TLS [31]) in the top layer by leveraging these quantum keys to perform key relay between two distant nodes, denoted as a source node s and a destination node d. A multi-hop path connecting s and d is first determined, and the end-to-end key originates at s. Between each adjacent node pair, the end-to-end key is encrypted with a quantum key generated by the bottom layer at one node and then transmitted to the next node via a classical channel, where the same quantum key is used for decryption. The one-time pad (OTP) algorithm is used to ensure information-theoretic security [32]. The process is repeated along the path until the end-to-end key reaches d. Finally, d acknowledges the success of the relay, and s supplies the end-to-end key to the requesting application.

B. Motivation

Due to the complicated process required to supply end-to-end keys, QKD networks struggle to provide an instant key supply service. At the bottom layer where QKD protocols are performed, secure key generation is governed by the probabilistic nature of quantum mechanics and the post-processing performance over the classical channel [33] and thus exhibits significant fluctuations, as amply demonstrated in operational QKD networks [34, Fig. 5]. The relaying process, which relays end-to-end keys hop-by-hop over a classical network, is exposed to the full spectrum of classical network dynamics in either the current overlay or the future integrated architecture [13]. Consequently, the key supply service relies on a cascade of dynamic processes, leading to highly fluctuating latency for key requests from applications.

While many existing works [17]–[22] focus on mitigating quantum layer performance fluctuations through key management or multipath key aggregation, they also introduce new fluctuations due to heterogeneous paths or strategy switching.



(a) Key supply latency. (b) Buffer size.

Fig. 1: Performance of the buffer-enabled key supply schemes.

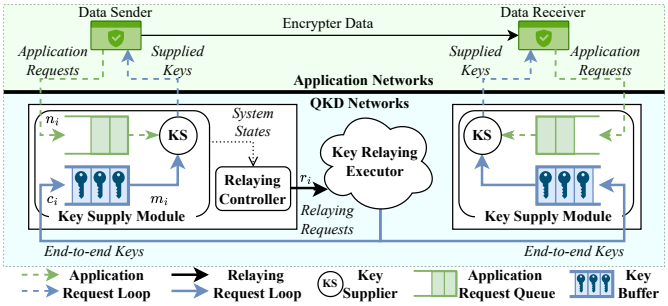


Fig. 2: Buffer-enabled key supply model in QKD networks.

Coupled with the inherent dynamics of the underlying classical network, the key relaying process and the key supply performance remain fluctuating. Fig. 1(a) depicts the test results obtained from a 14-node QKD network testbed (detailed in Sec. V-A), revealing extremely high and unstable key supply latency. The unstable latency leads to lags or interruptions in the key supply, resulting in degraded data transmission performance or compromised security, hindering the widespread application of QKD networks.

Some pioneering studies [24], [26] already consider the storable nature of keys and introduce end-to-end key buffering by relaying keys in advance, thereby providing a near-zero latency key supply service. As shown in Fig. 1(a), the state-of-the-art solution, DT-VQKP [26], as well as QuIKS (formally proposed in Sec. IV), demonstrate a decisive advantage of 0.3 ms versus 1100 ms in key supply latency and low fluctuation over direct relaying without a buffer. Despite this attractive advantage, DT-VQKP also requires a large buffer size of 107 KByte on average, as depicted in Fig. 1(b), due to heuristic thresholds and a bursty buffer filling strategy. Each buffered key must consume the same number of quantum keys on each link along the relaying path in QKD networks and cannot be reused for any other node pairs but the dedicated one. This fact leads to severe wastage of key resources, thereby degrading network performance and significantly hindering its value in practical QKD networks, which are key-limited with a typical quantum key generation rate of 79.3 Kbps [9]. However, it is not always necessary to have such a huge size. As also depicted in Fig. 1(b), by maintaining a buffer size of 1.6 KByte, the proposed QuIKS still achieves optimal key supply performance.

C. Design Goals

The above results reveal that a buffer can effectively mitigate fluctuations in key supply performance, but a large buffer poses a significant resource burden. Therefore, it is necessary to design a resource-efficient scheme while achieving the same key supply performance as in practical QKD networks. However, designing such a scheme remains challenging, primarily due to the stochastic nature of application requests and QKD networks, which simultaneously influence the buffer dynamics, and the lack of a quantitative evaluation for the performance-key consumption trade-off. These challenges, along with the lightweight requirement for practical algorithms, are the main reasons existing schemes employ heuristic control. By addressing these challenges, a practical buffering-based scheme should meet the following design goals:

1) **Guaranteed Performance**: To achieve continuous instant key supply for applications.
2) **Minimal Buffer Size**: To operate with the minimal possible key resource consumption.
3) **Lightweight and Adaptive Control**: To enable real-time, adaptive control with low computational overhead.

III. Buffer-enabled Key Supply Model

In this section, we first model the key supply with buffering as a special queuing system in QKD networks. Then, we analyze this system and derive theoretical conclusions to guide the key supply scheme proposed in the next section.

A. System Model

In a QKD network, considering a single data transmission direction for a pair of secure applications, key requests are always actively initiated by the sender, while the receiver passively generates key requests based on the received encrypted data. Therefore, the dynamics of the pair of key buffers are dominated by the sender-side one. Based on the basic key supply service in QKD networks, a buffer-enabled end-to-end key supply model is shown in Fig. 2. On both sides, application requests are decoupled from the key relaying process by a buffer-enabled key supply module, which is a parallel queuing system constituted by an application request queue and a key buffer. In this system, a key supplier operates according to the following rule: responding to a queuing application request by retrieving a key from the key buffer. Since this operation is simple, the service time is assumed to be zero. Additionally, to fill the key buffer, a relaying controller sends relaying requests according to the system states on the sender side.

Aligning with the existing QKD networks [35], the model assumes that key requests and relaying operate in fixed-size blocks. According to cutting-edge protocol design [36], the relaying process can operate reliably with no resource wastage. Thus, it assumes every relaying request can receive an end-to-end key. In this context, the relaying delay, which is the time between sending a relaying request and receiving the corresponding end-to-end key, can uniformly describe all QKD network fluctuations caused by key management schemes or

TABLE I: Notations

Notation	Definition
m_i	The key buffer size at the end of the i-th time slot.
n_i	The number of application requests arriving during the i-th time slot.
c_i	The number of keys arriving at the key buffer during the i-th time slot.
r_i	The number of relaying requests sent by the controller during i-th time slot.
ω_j	The probability that a relaying request receives the corresponding key after exactly j time slots.
K	The maximum possible delay of relaying requests in time slots, such that $\omega_j = 0$ for all $j > K$.
μ_n, μ_c	The long-term mean number of arrived application requests (n_i) and keys (c_i) per time slot, respectively. (i.e., $\mu_n = \lim_{N\to\infty} \frac{1}{N}\sum_{i=1}^{N} n_i$ and $\mu_c = \lim_{N\to\infty} \frac{1}{N}\sum_{i=1}^{N} c_i$).
$\Delta_{x,i}$	The change in the key buffer size m from the end of slot x to the end of slot i, i.e., $m_i - m_x$
$C_n(x)$	The auto-covariance function of the series $\{n_i\}$ at lag x.

classical networks. To precisely describe the changes in the key buffer, we introduce a time slot model, where each time slot has a length of T. On this basis, TABLE I presents the definitions of the used notations, and the state transition equations for the key buffer are as follows:

$$m_i = m_{i-1} - n_i + c_i, \tag{1}$$

$$c_i = \sum_{j=1}^{K} r_{i-j}\omega_{i-j,j}, \tag{2}$$

where $\omega_{i,j}$ is the probability that a relaying request sent in the i-th time slot experience a j-time slot delay, thus satisfying $\sum_{j=1}^{K} \omega_{i,j} = 1$. Eq. (2) reveals how QKD network fluctuations affect the arrival of keys. According to the service rule, at least one of the queues must be empty at the end of a time slot, as the service time is zero. Therefore, m_i physically indicates the key buffer size when it is positive, while its absolute value indicates a backlog of application requests vice versa.

B. Analysis of Relaying Control Strategy

To optimize key supply performance, it is equivalent to minimizing the queuing time of application requests before they are satisfied. When it is almost surely that the key buffer is non-empty when each application request arrives, the queuing time of application requests is 0. In the proposed model, it should satisfy that $m_i > 0$ for all i. According to Eq. (1), m_i effectively constitutes a random walk. Assuming that the long-term mean number of arrived application requests and keys per time slot exist, the asymptotic behavior of m_i satisfies the following equation:

$$\lim_{N\to\infty} \frac{m_N}{N} = \lim_{N\to\infty} \frac{1}{N}\sum_{i=1}^{N} c_i - \lim_{N\to\infty} \frac{1}{N}\sum_{i=1}^{N} n_i = \mu_c - \mu_n.$$

When $\mu_c < \mu_n$, which means that keys arrive slower than application requests, $\lim_{N\to\infty} m_N = -\infty$. In this case, since n_i is non-negative, $m_{i-1} \geq n_i$ is unachievable. When $\mu_c > \mu_n$, although there is $\lim_{N\to\infty} m_N = +\infty$ and thus $m_{i-1} \geq n_i$ definitely holds, the key buffer size will be unbounded. This result implies that the system cannot reach a steady state, rendering control meaningless, and an unbounded buffer contradicts our design goals.

This analysis reveals that for the system to operate in a stable regime with a bounded buffer, the primary principle for determining r_i must satisfy the condition $\mu_c = \mu_n$. Assume the relaying delay is stationary, i.e., $\omega_{i,j} = \omega_j$, meaning that network conditions do not exhibit significant changes over the considered timescale. If application requests depend only on their respective workloads, n_i and ω_j are independent. Given a reactive relaying control strategy of $r_i = n_i$, μ_c can be calculated according to Eq. (2) as follows:

$$\begin{aligned}\mu_c &= \lim_{N\to\infty} \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{K} n_{i-j}\omega_{i-j,j} \\ &= \sum_{j=1}^{K} \omega_j \lim_{N\to\infty} \frac{1}{N}\sum_{i=1}^{N} n_{i-j} = \sum_{j=1}^{K} \omega_j \mu_n.\end{aligned}$$

Recalling that $\sum_{j=1}^{K} \omega_j = 1$, there holds $\mu_c = \mu_n$.

Remark 1. *The reactive strategy, $r_i = n_i$, ensures long-term rate matching, which is the necessary condition to prevent both persistent key depletion and an unbounded buffer size.*

C. Analysis of the Optimal Key Buffer Size

The reactive strategy, $r_i = n_i$, prevents the first-order systematic drift in the key buffer. However, to achieve instant key supply, i.e., $m_i > 0$, it is necessary to account for short-term fluctuations, which requires a higher-order analysis of buffer size dynamics. By expanding Eq. (1), m_i can be expressed as follows:

$$\begin{aligned}m_i &= m_x - n_{x+1} + c_{x+1} - \cdots - n_i + c_i \\ &= m_x - \sum_{l=x+1}^{i} n_l + \sum_{l=x+1}^{i}\sum_{j=1}^{K} r_{l-j}\omega_{l-j,j}.\end{aligned} \tag{3}$$

Applying the reactive strategy $r_i = n_i$, substituting the stationary condition of $\omega_{l-j,j}$, and rearranging Eq. (3) yields:

$$\begin{aligned}\Delta_{x,i} &= -\sum_{j=1}^{K}\omega_j \sum_{l=x+1}^{i} n_l + \sum_{j=1}^{K}\omega_j \sum_{l=x+1}^{i} n_{l-j} \\ &= \sum_{j=1}^{K}\omega_j \sum_{l=x+1}^{i} n_{l-j} - n_l = \sum_{j=1}^{K}\omega_j S_{x,i}(j),\end{aligned} \tag{4}$$

where $S_{x,i}(j) = \sum_{l=x+1}^{i} n_{l-j} - n_l$.

Eq. (4) implies that under the reactive strategy, the change in the key buffer size is only related to the delay characteristics ω_j and K of QKD networks, and the application request arrival characteristics n_i. Further assuming the application request arrival is a wide-sense stationary (WSS) process over the considered timescale, which indicates that $E[n_i]$ exists and is constant, it is clear that $E[S_{x,i}(j)] = 0$, and thus $E[\Delta_{x,i}] = 0$. This result means that the key buffer size will fluctuate around a fixed value, and the distribution of $\Delta_{x,i}$ determines the key buffer size required to ensure $m_{i-1} \geq n_i$.

Examining the variance of $\Delta_{x,i}$, i.e., σ_Δ, it is effectively the second moment of $\Delta_{x,i}$, expressed as follows:

$$\sigma_\Delta^2 = E[(\sum_{j=1}^{K} \omega_j S_{x,i}(j))^2] = \sum_{j=1}^{K}\sum_{k=1}^{K} \omega_j\omega_k E[S_{x,i}(j)S_{x,i}(k)].$$

Let $S_{x,i}(j) = Z_x(j) - Z_i(j)$, where $Z_x(j) = \sum_{p=x-j+1}^{x} n_p$, and substituting it into $E[S_{x,i}(j)S_{x,i}(k)]$ yields:

$$\begin{aligned} E[S_{x,i}(j)S_{x,i}(k)] = E[Z_x(j)Z_x(k)] - E[Z_x(j)Z_i(k)] \\ - E[Z_i(j)Z_x(k)] + E[Z_i(j)Z_i(k)]. \end{aligned} \tag{5}$$

Since n_i is WSS, its auto-covariance depends only on the time difference and $\mu_n = E[n_i]$. Hence, $E[n_p n_q] = \mu_n^2 + C_n(p-q)$, and $E[Z_x(j)Z_i(k)]$ has the following form:

$$E[Z_x(j)Z_i(k)] = jk\mu_n^2 + \sum_{p=x-j+1}^{x}\sum_{q=i-k+1}^{i} C_n(p-q).$$

Further, assume that $i-x$ is sufficiently large compared to the typical correlation timescale of the application request. This is a reasonable assumption for many request models, which often exhibit short-range dependence or an exponentially decaying auto-correlation function, leading to $C_n(p-q) \approx 0$ for p near x and q near i. On this basis, summing all the four terms of Eq. (5) and simplifying yields:

$$E[S_{x,i}(j)S_{x,i}(k)] = 2\sum_{p=1}^{j}\sum_{q=1}^{k} C_n(p-q).$$

Let $\Lambda(j,k) = \sum_{p=1}^{j}\sum_{q=1}^{k} C_n(p-q)$ for simplification, σ_Δ^2 can be finally determined as follows:

$$\sigma_\Delta^2 = 2\sum_{j=1}^{K}\sum_{k=1}^{K} \omega_j\omega_k\Lambda(j,k). \tag{6}$$

While the exact distribution of $\Delta_{x,i}$ is unknown, its nature as a sum of many random variables justifies an approximation using the central limit theorem. Therefore, $\Delta_{x,i}$ is assumed to follow the normal distribution $\mathcal{N}(0, \sigma_\Delta^2)$. The objective of $m_i > 0$ is to dimension an initial buffer size L such that $L + \Delta_{x,i} > 0$ with a small tolerance ϵ. It leads to the condition $P(\Delta_{x,i} < -L) \le \epsilon$, and thus L can be given by the solution to $\Phi(-L/\sigma_\Delta) = \epsilon$, where Φ is the standard normal CDF.

Remark 2. *For a WSS application request arrival process under the reactive strategy, the initial buffer size L satisfying $\Phi(-L/\sigma_\Delta) = \epsilon$ provides an approximate guarantee against lagged key supply with a probability of at least $1-\epsilon$.*

IV. Lightweight Buffer Control Algorithm

A. Overview

Based on the preceding model and theoretical results, this section proposes a lightweight control algorithm, named QuIKS, for practice. The critical challenge is that the target buffer size needs to be calculated with unknown and dynamic system parameters, and QuIKS addresses this in a two-phase manner. It begins with a ***Probing and Adjusting*** phase to online estimate system parameters and establish a buffer guided by Remark 2. Subsequently, it enters a ***Stable Controlling*** phase, efficiently applying the reactive strategy as Remark 1 describes. To maintain adaptability to the non-stationary conditions in QKD networks, a lightweight mechanism reinitiates probing only when a significant buffer size drop is detected, ensuring robust and instant key supply.

Algorithm 1: Probing and Adjusting Phase

Input: The application requests record list $\{n\}$; The relaying delay record list $\{w\}$; Current time slot i_c.

1 Initiate $i_s \leftarrow i_c$, $\{n\} \leftarrow \{0\}$, $\{w\} \leftarrow \{0\}$, $K \leftarrow \infty$;
2 **while** $i_c < (i_s + (\alpha+1)K)$ **do** *// Probe parameters*
3 $N \leftarrow$ `Num_Of_Rcved_App_Requests()`;
4 $n_{i_c - i_s} \leftarrow N$ for list $\{n\}$;
5 Send N relaying requests;
6 **if** $i_c < (i_s + \alpha K)$ **then**
7 Send additional βN relaying requests;
8 **forall** w_i *in* $\{w\}$ **do**
9 $W \leftarrow$ `Num_Of_Rcved_Keys_With_Delay(`i`)`;
10 $w_i \leftarrow w_i + W$;
11 **if** *All requests sent in any time slot are satisfied* **then**
12 $K \leftarrow$ Index of the largest non-zero element of $\{w\}$;
13 Enter next time slot and $i_c = i_c + 1$;
14 Calculate σ_Δ^2 based on $\{n\}$, $\{w\}$, and K with Eq. (6);
15 $M \leftarrow$`Size_Of_Key_Buffer()`;
16 $d \leftarrow [5\sigma_\Delta] - M$;
17 **while** $d \neq 0$ **do** *// Adjust buffer size*
18 $N \leftarrow$ `Num_Of_Rcved_App_Requests()`;
19 Send $\max(0, N+d)$ relaying requests;
20 $d \leftarrow d + N - \max(0, N+d)$;
21 Enter next time slot and $i_c = i_c + 1$;
22 Enter Algorithm 2 steady controlling phase;

B. Probing and Adjusting Phase

Algorithm 1 illustrates the operation of the ***Probing and Adjusting*** phase of QuIKS. This phase comprises two processes: probing parameters (line 2) and adjusting buffer size (line 17).

The probing process lasts for $(\alpha+1)K$ time slots, during which relaying requests are sent at a rate of $\beta+1$ times the arrival rate of application requests in the first αK time slots. A larger α extends the observation window, perhaps yielding a more accurate estimation of $C_n(x)$. A larger β may result in a more reliable measurement of the maximum delay K and the delay distribution ω_j. In this process, the algorithm continuously records the number of received application requests in each time slot and the number of received keys corresponding to each delay value. Based on the records, after the relaying requests sent in a certain time slot are fully satisfied, K is updated by finding the maximum recorded relaying delay, which ensures that K is continuously revised monotonically. This process will definitely end as long as K is finite. Subsequently, $\{w\}$ is normalized to obtain each ω_j, and then σ_Δ can be calculated according to Eq. (6).

The adjusting process aims to steer the mean buffer level to a target size, which is set at $5\sigma_\Delta$ to ensure a small lagged supply probability ($\epsilon < 10^{-6}$) according to Remark 2. It first

Algorithm 2: Stable Controlling Phase

Input: The calculated σ_Δ^2; Current time slot i_c.

1 **while** *True* **do**
2 $M \leftarrow$`Size_Of_Key_Buffer();`
3 **if** $M < \sigma_\Delta$ **then**
4 Enter Algorithm 1 probing and adjusting phase;
5 **return**;
6 $N \leftarrow$`Num_Of_Rcved_App_Requests();`
7 Send N relaying requests;
8 Enter next time slot and $i_c = i_c + 1$;

calculates the required adjustment d, which is the difference between the target size and the current buffer size M. QuIKS uses the instantaneous buffer size M as an estimation of the pre-adjustment mean, which is justified because the last K time slots of the probing process intentionally wait for the over-requested keys to arrive, making the buffer enter a steady state. The algorithm then closes the difference d by either over-sending ($d > 0$) or under-sending ($d < 0$) relaying requests in several subsequent time slots. Ultimately, the buffer size is centered around the new target mean of $5\sigma_\Delta$ before entering the stable controlling phase.

C. Stable Controlling Phase

Algorithm 2 presents the operation of the ***Stable Controlling*** phase of QuIKS. In this phase, the task is to simply execute the reactive control strategy described as Remark 1, which always sends the same number of relaying requests as that of received application requests (line 7). While this simple strategy is highly efficient, QuIKS should remain adaptive to non-stationary network conditions without incurring significant overhead. To achieve this, it employs a lightweight, trigger-based mechanism that initiates a new probing phase only if the buffer size M drops below a conservative threshold of σ_Δ. Given that the buffer is maintained at a target of $5\sigma_\Delta$ and approximately follows $\mathcal{N}(5\sigma_\Delta, \sigma_\Delta)$, such a significant drop is a low-probability ($< 10^{-4}$) event under stable conditions, making it an efficient indicator of significant shifts in application requests or network characteristics. This ensures that while minor, harmless fluctuations are ignored, the system remains robust by maintaining a sufficient buffer ($M \geq \sigma_\Delta$) to provide a continuous instant key supply service.

D. Complexity Analysis

1) Space Complexity: Algorithm 1 collects the number of application requests and relay delays for $(\alpha+1)K$ time slots, while the peak buffer size is $(\alpha\beta - 1)K\mu_n$. Thus, its space complexity is $O(K\mu_n)$. The space complexity of Algorithm 2 is $O(1)$ owing to its simple design.

2) Time Complexity: For Algorithm 1, the complexity per time slot is $O(1)$, as the operations within each slot are limited to constant-time tasks such as comparisons and data recording. The primary computational overhead from calculating σ_Δ^2, which has an overall complexity of $O(K^2)$. This process involves computing $C_n(x)$ in $O(K^2)$ time and then constructing the $\Lambda(j,k)$ matrix. By employing the dynamic programming recurrence $\Lambda(j,k) = \Lambda(j-1,k) + \Lambda(j,k-1) - \Lambda(j-1,k-1) + C_n(j-k)$, the matrix construction and the subsequent double summation for σ_Δ^2 are both efficiently performed in $O(K^2)$. Thus, the overall time complexity of Algorithm 1 is $O(K^2)$, and the time complexity of Algorithm 2 is $O(1)$.

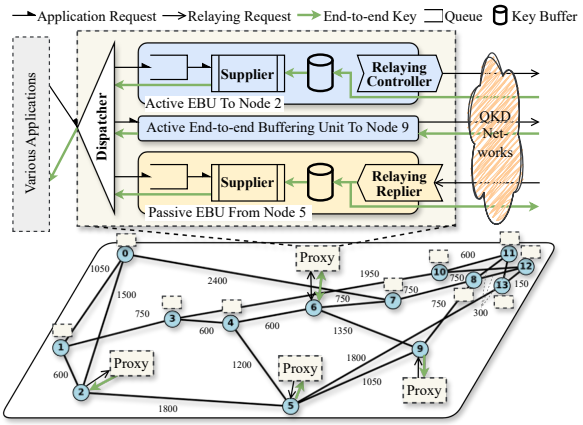


Fig. 3: Implementation of the testbed and key buffering architecture.

V. Experiment Results

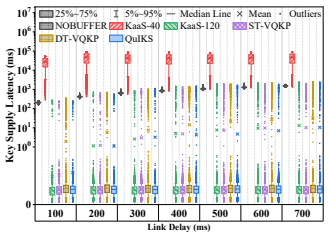
In this section, we introduce the implementation of the QKD network testbed and key buffering architecture. After that, we conduct the experiments to evaluate QuIKS.

A. Implementation

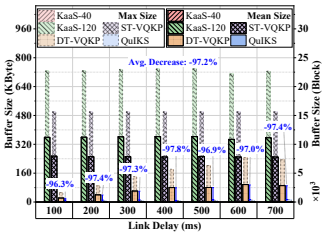
To evaluate QuIKS in a more realistic environment, we construct a QKD network testbed. We utilize 14 small-scale routers to form an NSFnet topology, with adjacent routers being directly connected via Ethernet cables, as shown in Fig. 3, where the numbers on the links represent the routing metrics. A forwarder daemon is deployed on each router, implementing the cutting-edge asynchronous key relay protocol [36]. Forwarders on adjacent routers establish TCP connections to form an overlay QKD network. Building upon this, we further design a proxy daemon to achieve application-transparent key buffering. This proxy acts as a middleware, communicating with applications and the forwarder via inter-process communication methods. Each proxy incorporates a request dispatcher and several end-to-end buffering units (EBU). The dispatcher redirects requests to the correct EBU based on the types-address tuple of encryption-destination or decryption-source, and creates an EBU when it does not exist. The EBU is responsible for request queuing, key supplying, and managing the key buffering with schemes like QuIKS.

B. Experiment Settings

In the experiments, we replay the data collected from commercial QKD devices (model: QuantumCTek QKD-PHA1250-S) to simulate realistic key generation between adjacent nodes. We use a 256-bit key block size to align with the key length of the AES algorithm, following [26], [36]. The comparison

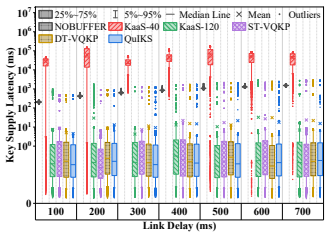


(a) Key supply latency.

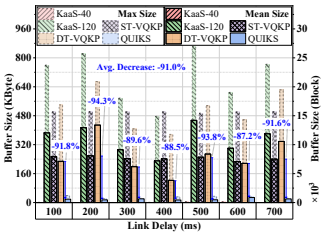


(b) Max and mean buffer size.

Fig. 4: Performance when application requests arrive as a Poisson process in the single-application scenario.



(a) Key supply latency.



(b) Max and mean buffer size.

Fig. 5: Performance when application requests arrive as a Poisson-Pareto process in the single-application scenario.

schemes include KaaS [24] with fixed relaying rates of 40 requests per second (rps) and 120 rps, as well as the state-of-the-art ST-VQKP and DT-VQKP, with their default parameters as specified in [26]. For QuIKS, we set $\alpha = \beta = 2$ as an empirical compromise. The duration of a time slot is set to 50 ms. Experimental variables include the application request rate, distribution, and total key demand, as well as the available quantum keys and link delay in the QKD network. Link delay is simulated using the `tc-netem` tool, following a normal distribution $\mathcal{N}(x, \frac{x}{10})$, where x is the mean link delay.

The considered performance metrics include buffer size, key supply latency, instant key supply ratio, and application completion ratio. The key supply latency refers to the time between an application sending a request and receiving the corresponding key. Note that the latency involves inherent processing time on the real system, but its optimization is beyond the scope of this paper. The instant key supply ratio is the proportion of requests with a key supply latency < 1 ms among all requests, where the threshold is to exclude the inherent processing time. The application completion ratio is the proportion of applications for which all requests are satisfied, relative to the total number of applications.

C. Performance in a Single-Application Scenario

In this scenario, experiments are conducted between node 0 and node 1 with only one application to evaluate the basic performance of QuIKS and comparison schemes. The application request rate is 50 rps, with the process being a Poisson process or a Poisson-Pareto Burst process (PPBP) [20]. In the composite PPBP process, the sending event follows a Poisson process with an arrival rate of 1, and the number of sent requests follows a Pareto distribution with a shape parameter of 2. The key demands of the application are 500 KByte, i.e., 15625 blocks, and quantum keys are sufficiently abundant in the network. The mean link delay ranges from 100 ms to 700 ms.

Fig. 4 presents the experimental results when application requests arrive as a Poisson process. As can be seen in Fig. 4(a), there are significant differences in key supply latency among NOBUFFER, KaaS-40, and other schemes. For the NOBUFFER scheme, the key supply latency is directly affected by the link delay. Due to a relaying rate lower than the application request rate, KaaS-40 always has an empty buffer as depicted in Fig. 4(b), resulting in high and unstable key supply latency. For other schemes like QuIKS, the key supply latency for 95% of requests consistently remains lower than 0.5 ms, with some high outliers caused by a temporal empty buffer at startup, no matter how the link delay changes. However, Fig. 4(b) shows that KaaS-120, ST-VQKP, and DT-VQKP have a large amount of buffer size, with the mean value up to 360 KByte achieved by KaaS-120. The best one of them, DT-VQKP, has a lower mean buffer size down to 22 KByte, which grows with the increase of the link delay because its relaying strategy employs a product of the application request rate and link delay, multiplied by a large factor. Even in this case, QuIKS's buffer size is still 97.1% lower than that of DT-VQKP on average, demonstrating its ultra-low key consumption and high resource efficiency.

Fig. 5 illustrates the results when application requests arrive as a PPBP process, where application requests arrive in a bursty pattern. In this case, the key supply latency increases and has a broader distribution as illustrated in Fig. 5(a). Despite this, for schemes except NOBUFFER and KaaS-40, the key supply latency for 95% of requests remains lower than 10 ms, where the increases are due to the bursty request pattern, independent of schemes. For the buffer size in this scenario, Fig. 5(b) depicts a slightly different result. In this scenario, among the schemes except QuIKS, the lowest mean buffer size is 122 KByte achieved by DT-VQKP when the link delay is 400 ms. ST-VQKP remains a larger but stable mean buffer size of around 250 KByte, as it is only related to the total number of requests, while KaaS-120 and DT-VQKP exhibit a random value of mean buffer size owing to the significant application request variations under the PPBP process. Although QuIKS's buffer size increases compared with that under the Poisson process, it still remains over 90% lower than that of the best one among other schemes on average, further demonstrating its advantage.

D. Deeper Analysis of the Buffer Size

Fig. 6 uses data with a link delay of 400 ms as an example to demonstrate the key buffer size per time slot, showcasing the control mechanisms of each scheme. These schemes, except KaaS-40 with zero buffer, can provide uninterrupted key supply. Hence, their application completion times are also annotated to reflect the application request characteristics

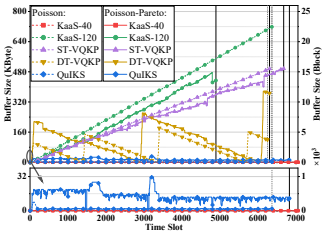


Fig. 6: Buffer size per time slot when delay is 400 ms.

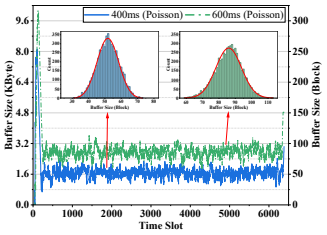


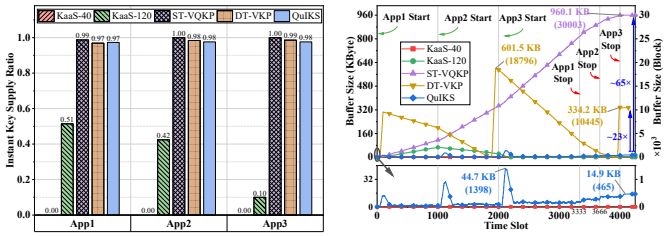
Fig. 7: Fitting of the buffer size distribution for QuIKS.

TABLE II: Comparison of parameters for QuIKS

Delay	Real σ_Δ	Est. $\hat{\sigma}_\Delta(-\sigma_\Delta)$	Fitted $\overline{\sigma}_\Delta(-\sigma_\Delta\%)$	R^2	χ^2_{red}
100 ms	3.4950	4.6596 **(+1.1646)**	3.4342 **(−1.7%)**	0.9887	2.4837
200 ms	5.0575	5.9812 **(+0.9237)**	4.8356 **(−4.4%)**	0.9709	4.4418
300 ms	5.6467	6.8012 **(+1.1545)**	5.4672 **(−3.2%)**	0.9785	2.6905
400 ms	6.5996	7.5748 **(+0.9752)**	6.6602 **(+0.9%)**	0.9823	2.2301
500 ms	7.6348	8.2602 **(+0.6254)**	7.6662 **(+0.4%)**	0.9904	1.0049
600 ms	7.9527	9.0477 **(+1.0950)**	7.8472 **(−1.3%)**	0.9869	1.2471
700 ms	8.9418	10.1152 **(+1.1734)**	8.9471 **(+0.1%)**	0.9846	1.3011

under either a Poisson or PPBP process, where these two types are represented by vertical gray dashed and black solid lines, respectively. Under the PPBP process, the significant variation in application completion times is attributed to the high variation in request rate per time slot. As shown in Fig. 6, the buffer size of KaaS-120, with its constant relaying rate, is primarily affected by application completion time, while that of DT-VQKP, with a request rate-based bursty relaying method, is governed by the request rate at the moment of relaying. Consequently, both exhibit larger and more fluctuating buffers as previously shown in Fig. 5(b). Furthermore, it can be seen that QuIKS's buffer size under the PPBP process is significantly larger than that under the Poisson process, with only a few occasional probing and adjusting phases during operation. This indicates that QuIKS successfully captures the drastic variations in bursty application requests and thus possesses high adaptiveness.

Fig. 7 illustrates the buffer size and the fitted distribution of QuIKS under a Poisson process, with link delays of 400 ms and 600 ms. Discarding the initial several time slots of the probing and adjusting phase, QuIKS can maintain an ultra-low buffer size under both conditions, and its distribution intuitively fits a normal distribution. This observation is validated in TABLE II, where the fitted $\overline{\sigma}_\Delta$ deviates from the real σ_Δ of the buffer size by less than 5% and exhibits a high coefficient of determination (R^2) and low reduced chi-square χ^2_{red}. These results mean that the distribution of buffer size statistically follows a normal distribution, which confirms the accuracy of Remark 2. On this basis, the $\hat{\sigma}_\Delta$ estimated according to Algorithm 1 exhibits a stable positive difference of around 1 to the real σ_Δ, meaning that it can capture the most important trends in buffer size variation when link delay changes, despite the bias due to lightweight designs. Also, the overestimation provides a conservative margin in practice, while small enough to keep the superiority of QuIKS over other schemes.



(a) Instant key supply ratio.

(b) Buffer size per time slot.

Fig. 8: Performance in the multiple-application scenario.

E. Performance in a Multiple-Application Scenario

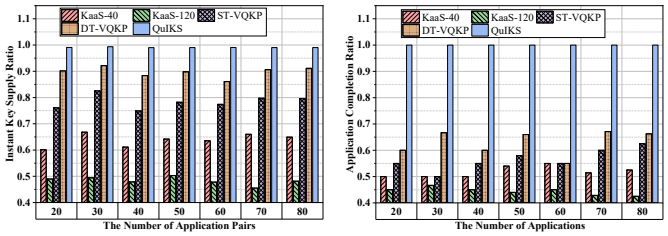
In this scenario, experiments are conducted between node 3 and node 7, following the path of 3-4-6-7, to evaluate how multiple applications affect the buffer. Three applications start in sequence, each with a request rate of 75 rps in a Poisson process and key demands of 400, 320, and 240 KByte, respectively. The mean delay for each link is set to 200 ms, and quantum keys are sufficiently abundant.

Fig. 8 presents the experiment results, where KaaS exhibits a poor instant key supply ratio in Fig. 8(a). This is attributed to its fixed relaying rate, which cannot cope with dynamic request rates. As shown in Fig. 8(b), when there is only one application, KaaS-120 can handle and accumulate some keys in the buffer. But as the subsequent applications join, the total request rate (>150) exceeds its relaying rate, making the buffer eventually be exhausted and resulting in a significant drop in performance after time slot 2000. For ST-VQKP, its relaying rate is always twice the request rate, so there is a clear change in buffer size growth when the application starts or stops. Hence, it leads to an optimal instant key supply ratio but an extremely large buffer size. However, as illustrated in Fig. 8(a), both DT-VQKP and QuIKS achieve a comparable ratio of over 0.97 while reducing the buffer size. Notably, QuIKS reduces buffer size at the end by $65\times$ and $23\times$ over ST-VQKP and DT-VQKP, respectively, with a temporally large size occurring only when applications join. Therefore, QuIKS can adapt to multiple applications and still maintain an instant key supply with ultra-low key resource consumption.

F. Performance in a Key-Limited Scenario

In this scenario, to evaluate the performance of various schemes when faced with numerous applications and limited quantum keys, experiments are conducted across the entire NSFnet topology as depicted in Fig. 3 with the shortest path routing. Applications are generated with random positions, their quantity ranging from 20 to 80. Each of them starts randomly within [0, 150] seconds, and has a request rate of 10 rps in a Poisson process and key demands of 50 KByte. The mean delay for each link is set to 200 ms, and quantum keys are merely 10% higher than the minimum requirements.

Fig. 9(a) illustrates the instant key supply ratio of all schemes. In this case, KaaS-40 is better than KaaS-120 because KaaS does not stop relaying, and the latter has a higher relaying rate, which can easily exhaust the quantum keys in



(a) Instant key supply ratio. (b) Application completion ratio.

Fig. 9: Performance in the key-limited scenario.

early-starting buffers. Without available quantum keys, the key relaying necessary for key supply fails, resulting in a low application completion ratio, as depicted in Fig. 9(b). In this key-limited scenario, lower quantum key consumption actually leads to optimal performance. ST-VQKP sends relaying requests at double the rate of application requests and stops with them. Therefore, it consumes fewer quantum keys than KaaS, ensuring that late-starting buffers have sufficient quantum keys available, which leads to a higher instant key supply ratio and application completion ratio. Similar to the results in previous scenarios, DT-VQKP exhibits sawtooth-like buffer fluctuations that do not continuously increase, thereby consuming much fewer quantum keys and achieving 90% instant key supply ratio as shown in Fig. 9(a). However, DT-VQKP still requires a large buffer size, leading to key supply interruptions for many applications nearing completion, as shown in Fig. 9(b). QuIKS achieves a 100% application completion ratio and a near-optimal 99% instant key supply ratio with an ultra-low buffer size and quantum key consumption. The remaining 1% is attributed to the inherent queuing that occurs when a buffer is initializing. The results further demonstrate that QuIKS offers an excellent trade-off between performance and key resource consumption, meeting the design goals.

VI. Related Work

Path-level key management. A primary field of research focuses on maximizing key utilization efficiency on a single path to provide sufficient key resources for continuous instant key supply. Protocol-level innovations, such as key caching in IC-QKD [37] and precise key and request management in AKRP [36], aim to reduce key consumption during the relaying process. From a resource allocation perspective, some works [17], [18] introduce management schemes that prioritize critical requests to enhance overall relaying performance on each node. However, these path-level optimizations are fundamentally constrained by the finite key resources available on that single path. They cannot aggregate key resources in the network, making them insufficient when a path's intrinsic quantum keys cannot meet escalating demands.

Network-wide path selection. To overcome the limitations of single-path schemes, numerous studies have explored dynamic path selection to aggregate key resources across the QKD network. These routing schemes dynamically discover key-abundant paths to serve requests [19], [22], [38], [39]. Some advanced approaches also incorporate classical network metrics, such as latency and bandwidth, to bypass underperforming paths [19], [20]. However, while solving for quantum key availability, these schemes introduce new performance volatility. The switching between paths with heterogeneous characteristics (e.g., latency, hops), coupled with the unpredictable performance of classical networks, where key relaying is executed, leads to an unstable key distribution. Consequently, they fail to guarantee the stable and predictable relaying performance required for instant key supply.

Service-layer decoupling via buffering. To fundamentally address the problem, end-to-end buffering has emerged as a promising paradigm to decouple key supply services from network volatility. The concept is pioneered in KaaS [24], which uses static scheduling to pre-fill buffers. Subsequent works have refined this idea, from immediately relaying keys to construct application-specific buffers [40], to hierarchical slicing for finer-grained services [25]. More recently, research has begun to explore dynamic buffering strategies [26]. However, while establishing the benefits of buffering, these existing schemes predominantly rely on static or heuristic methods for determining buffer sizes and relaying strategies. A systematic framework that balances the trade-offs between key supply performance and overhead, such as key consumption, remains a critical and under-addressed research gap.

VII. Conclusion

In this paper, we proposed an instant key supply scheme based on adaptive buffering in QKD networks, named QuIKS, to address the dilemma between key resource consumption and key supply performance. The challenges stem from the fundamental difficulty of modeling stochastic buffer dynamics and the lack of a quantitative framework. To overcome these challenges, we established a novel theoretical model to analyze the impact of stochastic application requests and the QKD network on buffer dynamics, and then we quantified the mathematical relationship between key supply performance and buffer size. Guided by this model, we designed a lightweight two-phase buffer control algorithm to provide instant key supply services with a theory-guided ultra-low key buffer size. Extensive experiments on a real-world QKD testbed demonstrate that QuIKS achieves a near-zero key supply latency while reducing key buffer size by more than 90%, thus minimizing key resource consumption and paving the way for high-performance and resource-efficient QKD networks.

Acknowledgement

This work is supported in part by the National Natural Science Foundation of China under Grant No. 62572450, Grant No. 62402466, and Grant No. 62501562, the Innovation Program for Quantum Science and Technology under Grant No. 2021ZD0301301, the Youth Innovation Promotion Association Chinese Academy of Sciences under Grant No. Y202093, and Japan Society for the Promotion of Science (JSPS) KAKENHI under Grant No. 23K28070.

REFERENCES

[1] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," in *Proceedings of IEEE International Conference on Computers Systems and Signal Processing*, 1984, pp. 175–179.

[2] H.-K. Lo and H. F. Chau, "Unconditional security of quantum key distribution over arbitrarily long distances," *Science*, vol. 283, no. 5410, pp. 2050–2056, 1999.

[3] C. Portmann and R. Renner, "Security in quantum cryptography," *Reviews of Modern Physics*, vol. 94, no. 2, p. 025008, 2022.

[4] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021.

[5] K. H. Shakib, M. Rahman, M. Islam, and M. Chowdhury, "Impersonation attack using quantum Shor's algorithm against blockchain-based vehicular ad-hoc network," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[6] C. Elliott, D. Pearson, and G. Troxel, "Quantum cryptography in practice," in *Proceedings of ACM SIGCOMM*, 2003, pp. 227–238.

[7] M. Peev, C. Pacher, R. Alléaume, C. Barreiro, J. Bouda, W. Boxleitner, T. Debuisschert, E. Diamanti, M. Dianati, J. Dynes *et al.*, "The SECOQC quantum key distribution network in Vienna," *New Journal of Physics*, vol. 11, no. 7, p. 075001, 2009.

[8] M. Sasaki, M. Fujiwara, H. Ishizuka, W. Klaus, K. Wakui, M. Takeoka, S. Miki, T. Yamashita, Z. Wang, A. Tanaka *et al.*, "Field test of quantum key distribution in the Tokyo QKD network," *Optics Express*, vol. 19, no. 11, pp. 10 387–10 409, 2011.

[9] Y.-A. Chen, Q. Zhang, T.-Y. Chen, W.-Q. Cai, S.-K. Liao, J. Zhang, K. Chen, J. Yin, J.-G. Ren, Z. Chen *et al.*, "An integrated space-to-ground quantum communication network over 4,600 kilometres," *Nature*, vol. 589, no. 7841, pp. 214–219, 2021.

[10] B. Huttner, R. Alléaume, E. Diamanti, F. Fröwis, P. Grangier, H. Hübel, V. Martin, A. Poppe, J. A. Slater, T. Spiller *et al.*, "Long-range QKD without trusted nodes is not possible with current technology," *npj Quantum Information*, vol. 8, no. 1, p. 108, 2022.

[11] Y. Cao, Y. Zhao, Q. Wang, J. Zhang, S. X. Ng, and L. Hanzo, "The evolution of quantum key distribution networks: on the road to the Qinternet," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 2, pp. 839–894, 2022.

[12] Z. Li, K. Xue, J. Li, L. Chen, R. Li, Z. Wang, N. Yu, D. S. L. Wei, Q. Sun, and J. Lu, "Entanglement-assisted quantum networks: mechanics, enabling technologies, challenges, and research directions," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2133–2189, 2023.

[13] J. Li, P. Zheng, Z. Li, K. Xue, Z. Xie, N. Yu, Q. Sun, and J. Lu, "Integration of quantum key distribution networks and classical networks: An evolution perspective," *IEEE Network*, vol. 39, no. 3, pp. 180–187, 2025.

[14] L. Zhang, W. Li, J. Pan, Y. Lu, W. Li, Z.-P. Li, Y. Huang, X. Ma, F. Xu, and J.-W. Pan, "Experimental mode-pairing quantum key distribution surpassing the repeaterless bound," *Physical Review X*, vol. 15, no. 2, p. 021037, 2025.

[15] X. Liu, S. Zhao, Y. Cui, and X. Wang, "Figret: Fine-grained robustness-enhanced traffic engineering," in *Proceedings of ACM SIGCOMM*, 2024, pp. 117–135.

[16] K. S. Namjoshi, S. Gheissi, and K. Sabnani, "Algorithms for in-place, consistent network update," in *Proceedings of ACM SIGCOMM*, 2024, pp. 244–257.

[17] H. Zhou, K. Lv, L. Huang, and X. Ma, "Quantum network: Security assessment and key management," *IEEE/ACM Transactions on Networking*, vol. 30, no. 3, pp. 1328–1339, 2022.

[18] J. Li, P. Zheng, Z. Li, Y. Yang, N. Yu, Q. Sun, and J. Lu, "Decentralized key management and service in quantum key distribution networks: An experimental implementation," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 8, pp. 2782–2797, 2025.

[19] M. Mehic, P. Fazio, S. Rass, O. Maurhart, M. Peev, A. Poppe, J. Rozhon, M. Niemiec, and M. Voznak, "A novel approach to quality-of-service provisioning in trusted relay quantum key distribution networks," *IEEE/ACM Transactions on Networking*, vol. 28, no. 1, pp. 168–181, 2019.

[20] M. S. Akhtar, K. G, V. B, and A. Sinha, "Fast and secure routing algorithms for quantum key distribution networks," *IEEE/ACM Transactions on Networking*, vol. 31, no. 5, pp. 2281–2296, 2023.

[21] Q. Zhang, O. Ayoub, A. Gatto, J. Wu, F. Musumeci, and M. Tornatore, "Routing, channel, key-rate, and time-slot assignment for qkd in optical networks," *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 148–160, 2023.

[22] P. Zheng, J. Li, Z. Li, K. Xue, N. Yu, R. Li, Q. Sun, and J. Lu, "An efficient and robust resource allocation method for quantum key distribution networks," *IEEE Transactions on Network and Service Management*, vol. 22, no. 5, pp. 4083–4095, 2025.

[23] Y. Zhang, H. Zhang, P. Cong, and W. Wang, "ROND: Rethinking overlay network design with underlay network awareness," *Proceedings of the ACM on Networking*, vol. 2, no. CoNEXT2, pp. 1–22, 2024.

[24] Y. Cao, Y. Zhao, J. Wang, X. Yu, Z. Ma, and J. Zhang, "KaaS: Key as a service over quantum key distribution integrated optical networks," *IEEE Communications Magazine*, vol. 57, no. 5, pp. 152–159, 2019.

[25] Q. Zhu, X. Yu, Y. Zhao, A. Nag, and J. Zhang, "QKD key provisioning with multi-level pool slicing for end-to-end security services in optical networks," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 2, pp. 2153–2169, 2023.

[26] C. Stan, D. Verchere, J. J. V. Olmos, I. Tafur Monroy, and S. Rommel, "Dynamic-threshold-based pre-relaying for enhanced key allocation in quantum-secured networks," *Journal of Optical Communications and Networking*, vol. 17, no. 3, pp. 233–248, 2025.

[27] H. Du, T. K. Paraiso, M. Pittaluga, Y. S. Lo, J. A. Dolphin, and A. J. Shields, "Twin-field quantum key distribution with optical injection locking and phase encoding on-chip," *Optica*, vol. 11, no. 10, pp. 1385–1390, 2024.

[28] S.-C. Zhuang, B. Li, M.-Y. Zheng, Y.-X. Zeng, H.-N. Wu, G.-B. Li, Q. Yao, X.-P. Xie, Y.-H. Li, H. Qin *et al.*, "Ultrabright entanglement based quantum key distribution over a 404 km optical fiber," *Physical Review Letters*, vol. 134, no. 23, p. 230801, 2025.

[29] "Quantum key distribution networks - Functional architecture," International Telecommunication Union, 2020, Recommendation Y.3802, accessed on: Dec. 2025. [Online]. Available: https://www.itu.int/rec/T-REC-Y.3802-202012-I/en

[30] X. Gao, K. Xue, J. Li, Z. Li, J. Wu, N. Yu, Q. Sun, and J. Lu, "IPSeQ: A security-enhanced ipsec protocol integrated with quantum key distribution," *IEEE Communications Magazine*, vol. 63, no. 9, pp. 148–155, 2025.

[31] C. R. Garcia, A. C. Aguilera, C. Stan, J. J. Vegas, S. Rommel, and I. T. Monroy, "Enhanced network security protocols for the quantum era: Combining classical and post-quantum cryptography, and quantum key distribution," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 8, pp. 2765–2781, 2025.

[32] C. E. Shannon, "Communication theory of secrecy systems," *The Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.

[33] M. Mehic, O. Maurhart, S. Rass, D. Komosny, F. Rezac, and M. Voznak, "Analysis of the public channel of quantum key distribution link," *IEEE Journal of Quantum Electronics*, vol. 53, no. 5, pp. 1–8, 2017.

[34] V. Martin, J. Brito, L. Ortíz, R. Mendez, J. Buruaga, R. Vicente, A. Sebastián-Lombraña, D. Rincón, F. Pérez, C. Sánchez *et al.*, "MadQCI: a heterogeneous and scalable SDN-QKD network deployed in production facilities," *npj Quantum Information*, vol. 10, no. 1, p. 80, 2024.

[35] E. Dervisevic, A. Tankovic, E. Fazel, R. Kompella, P. Fazio, M. Voznak, and M. Mehic, "Quantum key distribution networks-key management: A survey," *ACM Computing Surveys*, vol. 57, no. 10, pp. 1–36, 2025.

[36] Y. Chen, J. Li, Z. Li, K. Xue, N. Yu, Q. Sun, and J. Lu, "An asynchronous key relay protocol design for large-scale quantum key distribution networks," *IEEE Transactions on Networking*, vol. 33, no. 6, pp. 3024–3039, 2025.

[37] Q. Zhang, O. Ayoub, J. Wu, X. Lin, and M. Tornatore, "IC-QKD: An information-centric quantum key distribution network," *IEEE Communications Magazine*, vol. 61, no. 12, pp. 148–154, 2023.

[38] S. Xu, Y. Zhao, L. Huang, and C. Qiao, "Routing and photon source provisioning in quantum key distribution networks," in *Proceedings of IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2024, pp. 1411–1420.

[39] M. Wenning, J. Berl, T. Fehenberger, and C. Mas-Machuca, "Comparison of distributed and centralized quantum key management systems for meshed QKD networks," *Journal of Optical Communications and Networking*, vol. 17, no. 2, pp. A224–A233, 2025.

[40] X. Yu, X. Liu, Y. Liu, A. Nag, X. Zou, Y. Zhao, and J. Zhang, "Multi-path-based quasi-real-time key provisioning in quantum-key-distribution enabled optical networks (QKD-ON)," *Optics Express*, vol. 29, no. 14, pp. 21 225–21 239, 2021.